

Gradient Descent & Optimizer Variants

Central Question: Given a loss function, a starting point, and only local gradient information, how can we reliably descend toward a minimum?

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla f(\theta_t)$$

Core Thesis: Gradient descent moves opposite the direction of steepest local increase. Optimizer variants modify how gradients are *estimated, remembered, scaled, regularized, or evaluated*.

💡 **Roadmap:** Vector Calculus Foundation → Gradient Descent Derivation → Curvature & Stability → 8 Optimizer Variants → NumPy CNN on MNIST.

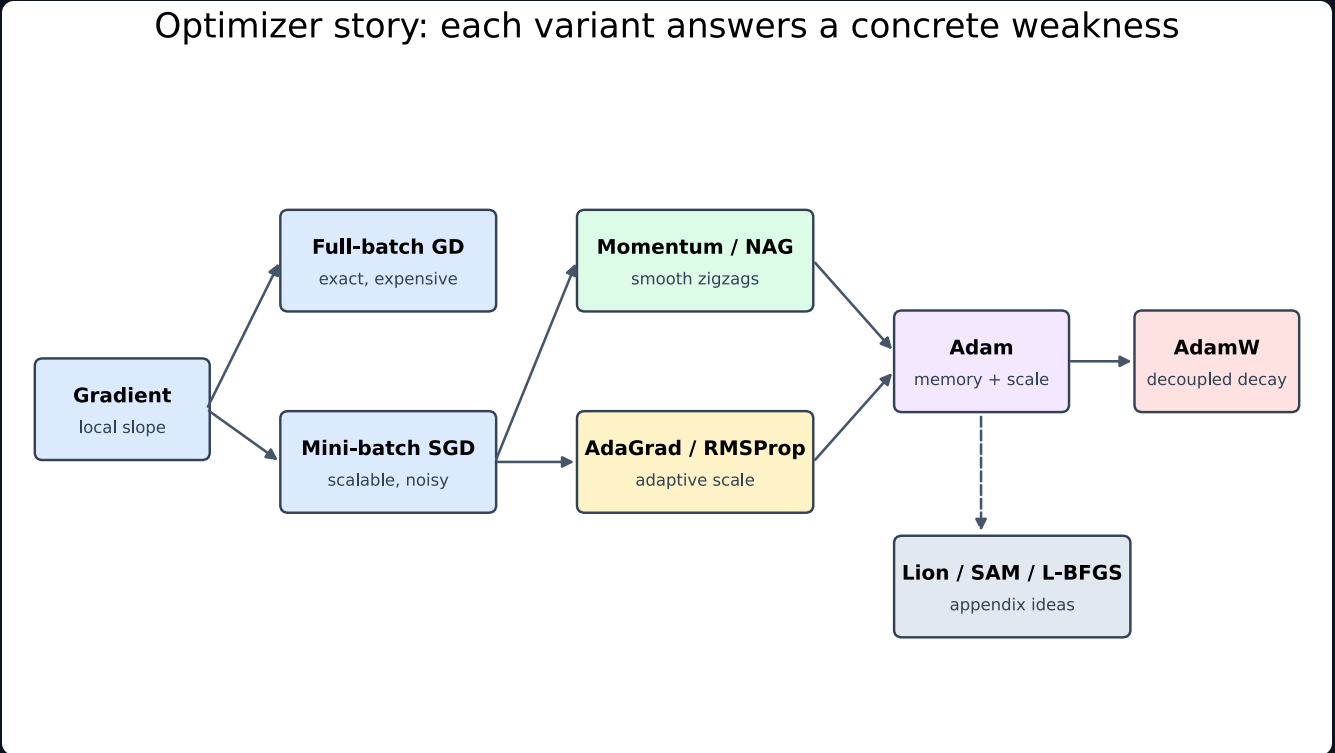


Figure 08: The Family Map of Modern Optimizers

The Gradient as a Vector Field

For a scalar-valued multivariable function $f(x, y)$, the gradient $\nabla f(x, y)$ packages all partial derivatives into a vector in parameter space:

$$f(x, y) = x^2 + y^2 \implies \nabla f(x, y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

- At $(1, 2)$: $\nabla f(1, 2) = [2, 4]^T$.
- The gradient vector points in the direction of **steepest local increase**.
- Its magnitude $\|\nabla f\|$ gives the maximal instantaneous rate of change.

📌 *Key distinction:* The gradient lives in the parameter space (input plane), orthogonal to the contour level curves.

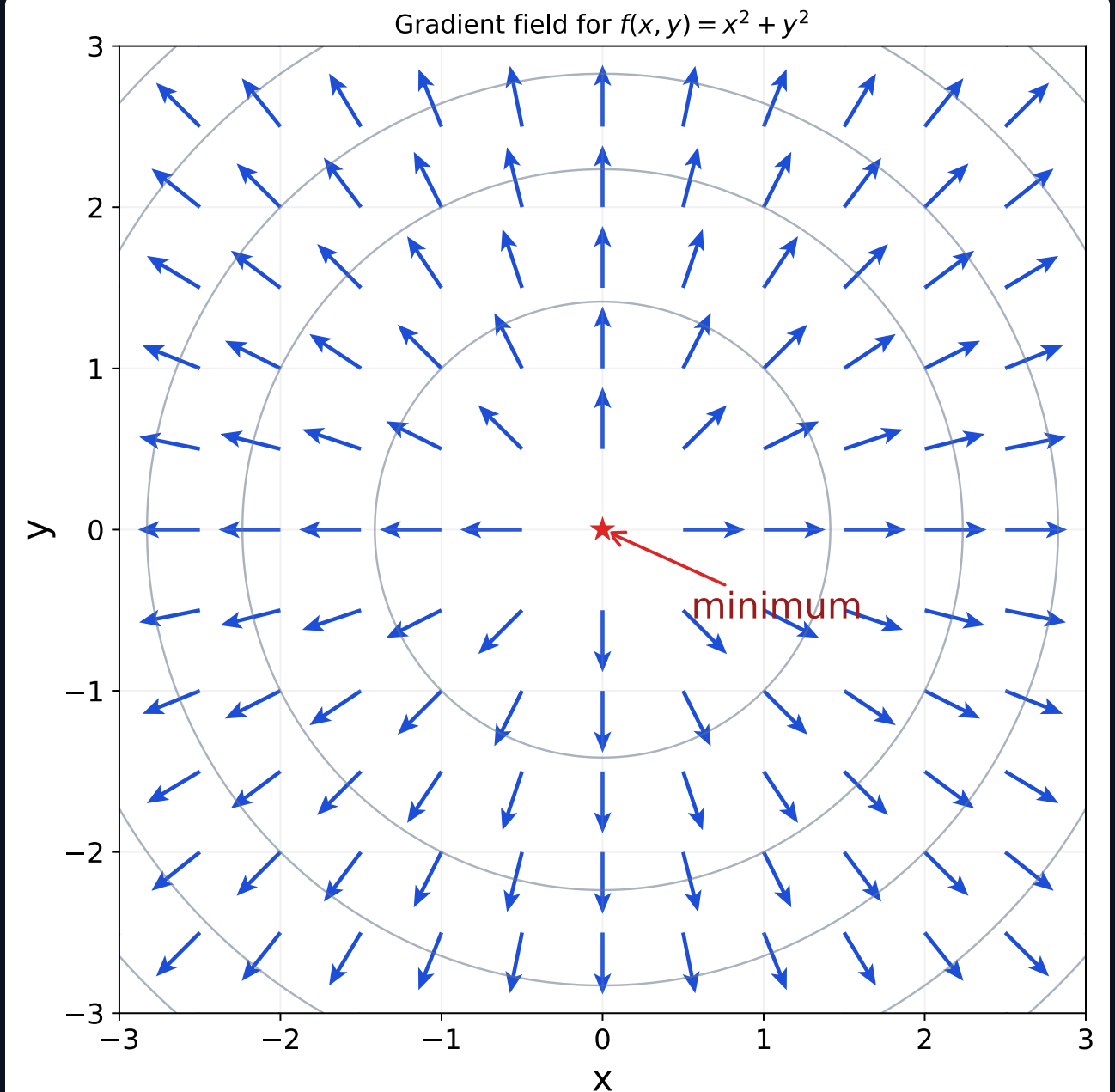


Figure 01: Vector field $\nabla f(x, y)$ pointing radially outward

3D Loss Landscapes & The Spherical Bowl

Visualizing the graph $z = f(x, y) = x^2 + y^2$:

- Convex quadratic with a unique global minimum at $(0, 0)$ where $\nabla f(0, 0) = \mathbf{0}$.
- Isotropic curvature: the rate of change is symmetric in all directions ($H = 2I$).
- In high-dimensional machine learning, we navigate surfaces with millions of parameters using only local gradient evaluations.

 **Objective:** Reach the bottom of the bowl using the fewest and most stable steps.

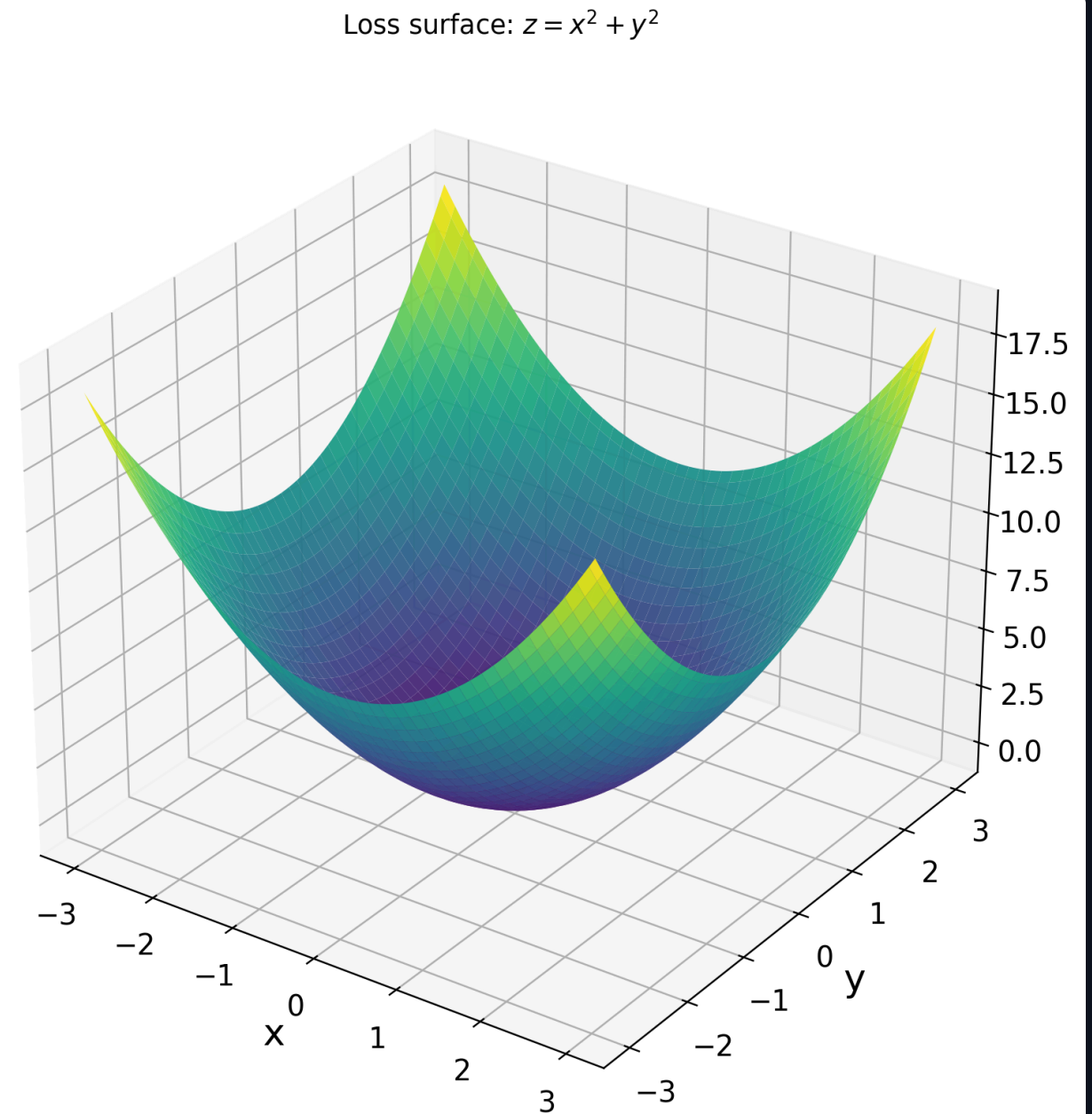


Figure 02: 3D bowl representation with global minimum at origin

Why the Negative Gradient is Downhill

1. Directional Derivative: Along unit vector u ($|u|=1$):

$$D_u f = \nabla f \cdot u = |\nabla f| \cos(\theta)$$

By Cauchy-Schwarz, $D_u f$ is minimal (most negative) when $\cos(\theta) = -1$, meaning u points directly **opposite** ∇f .

2. First-order Taylor Expansion:

$$f(\theta - \eta \nabla f) \approx f(\theta) - \eta |\nabla f|$$

For any small $\eta > 0$ and nonzero gradient, the objective is guaranteed to decrease locally!

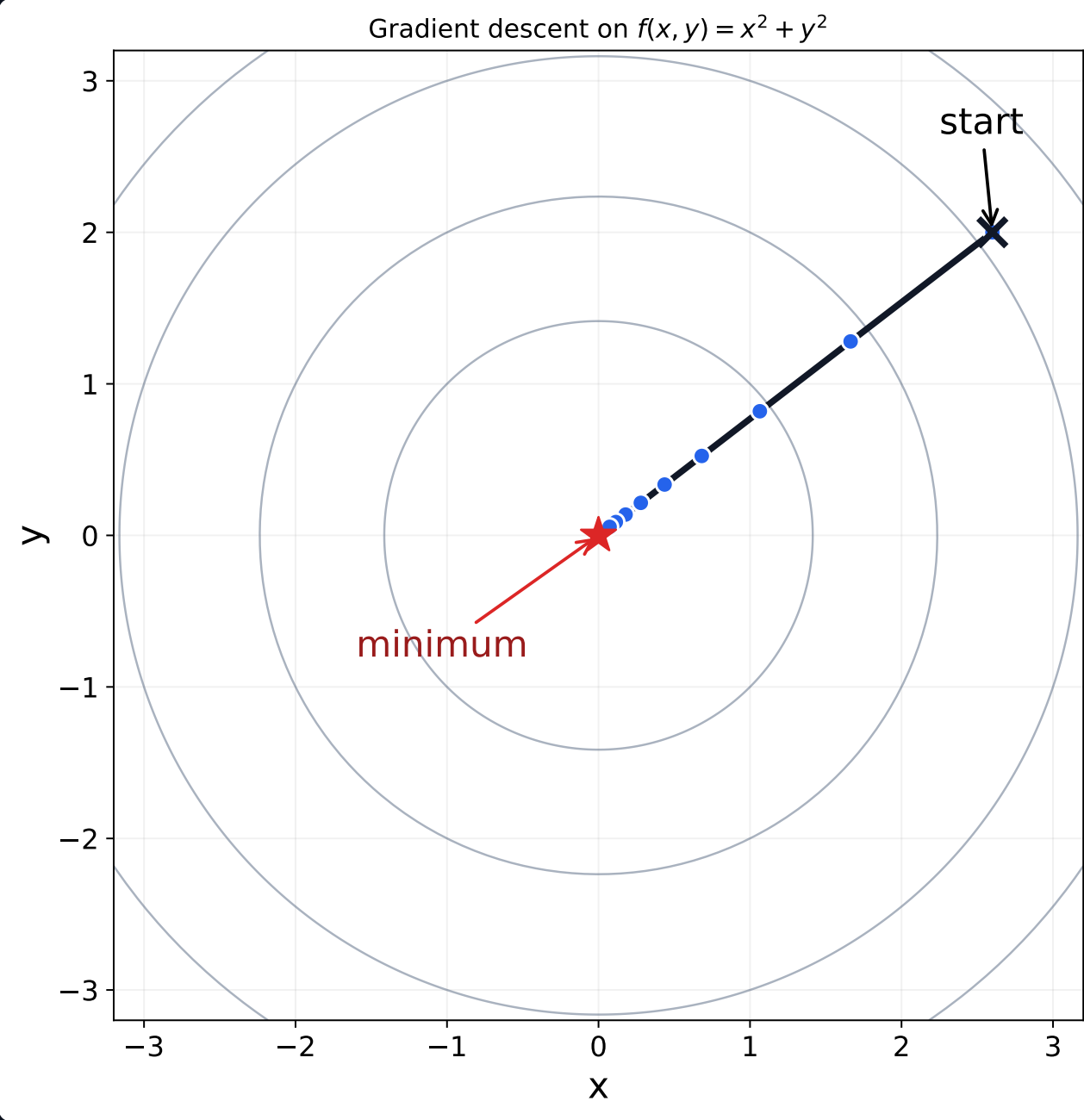


Figure 03: Step-by-step negative gradient descent trajectory

First Step Hand Calculation & Dynamics

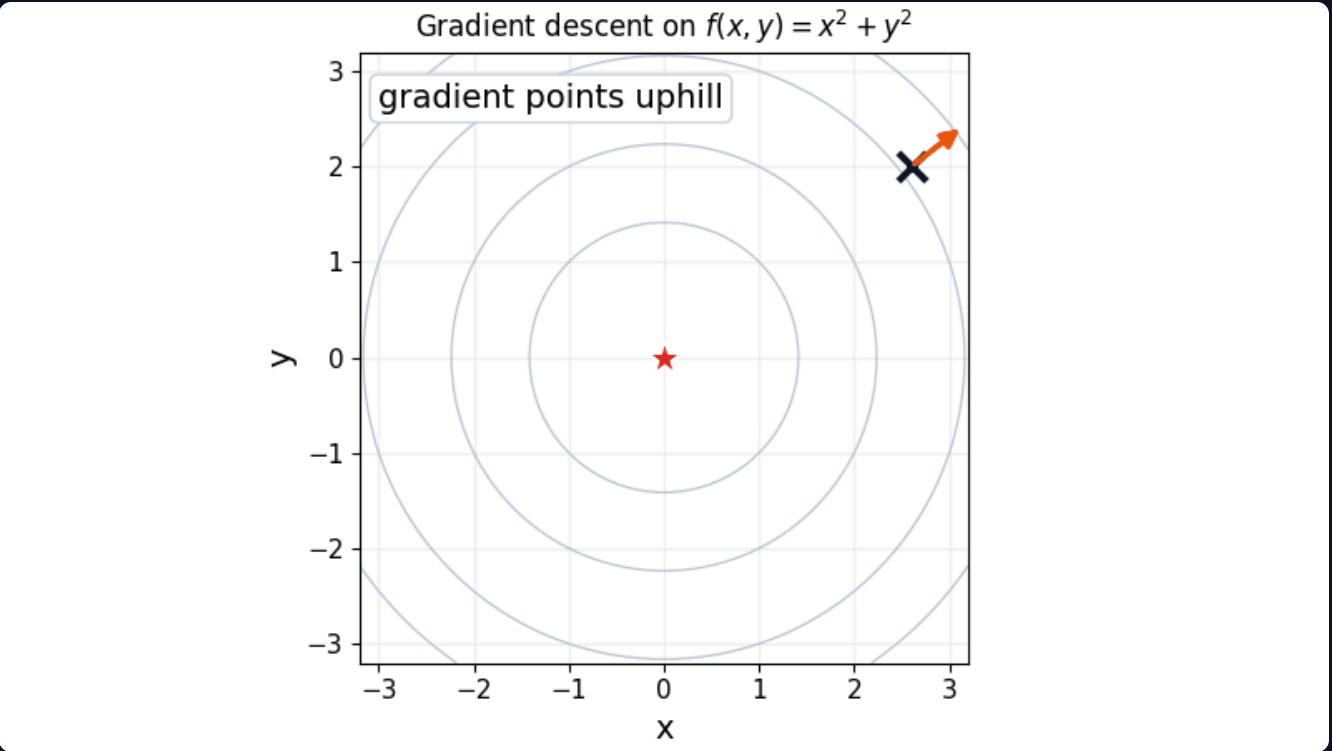
Classroom Verification Exercise:

- Start: $\theta_0 = (1, 2)$, loss $f(\theta_0) = 1^2 + 2^2 = 5.0$.
- Gradient: $\nabla f(1, 2) = (2, 4)$.
- Learning rate: $\eta = 0.1$.

```
$$\theta_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 0.1 \\ \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 1.6 \end{bmatrix}$$$
```

New loss value: $0.8^2 + 1.6^2 = 0.64 + 2.56 = \mathbf{3.2} < 5.0$.

✓ The algebraic formula, vector step, and contour animation match precisely!



Animated: Gradient Descent trajectory converging to minimum

The Ill-Conditioned Quadratic $x^2 + 25y^2$

Consider the anisotropic quadratic landscape:

```
$$f(x, y) = x^2 + 25y^2, \quad \nabla f(x, y) = \begin{bmatrix} 2x \\ 50y \end{bmatrix}, \quad H = \begin{bmatrix} 2 & 0 \\ 0 & 50 \end{bmatrix} \\ \end{bmatrix}$$$
```

- **Condition number:** $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{50}{2} = \mathbf{25}$.
- Curvature is 25 times steeper along the y -axis than along the x -axis.
- **Coordinate update:** $x_{t+1} = (1 - 2\eta)x_t$, $y_{t+1} = (1 - 50\eta)y_t$.
- **Stability condition:** $|1 - 50\eta| < 1 \implies \mathbf{0 < \eta < 0.04}$.

⚠ Common Misconception: This is a convex problem with ONE global minimum at $(0, 0)$. Zigzagging and divergence here are caused by curvature disparity, NOT local minima traps!

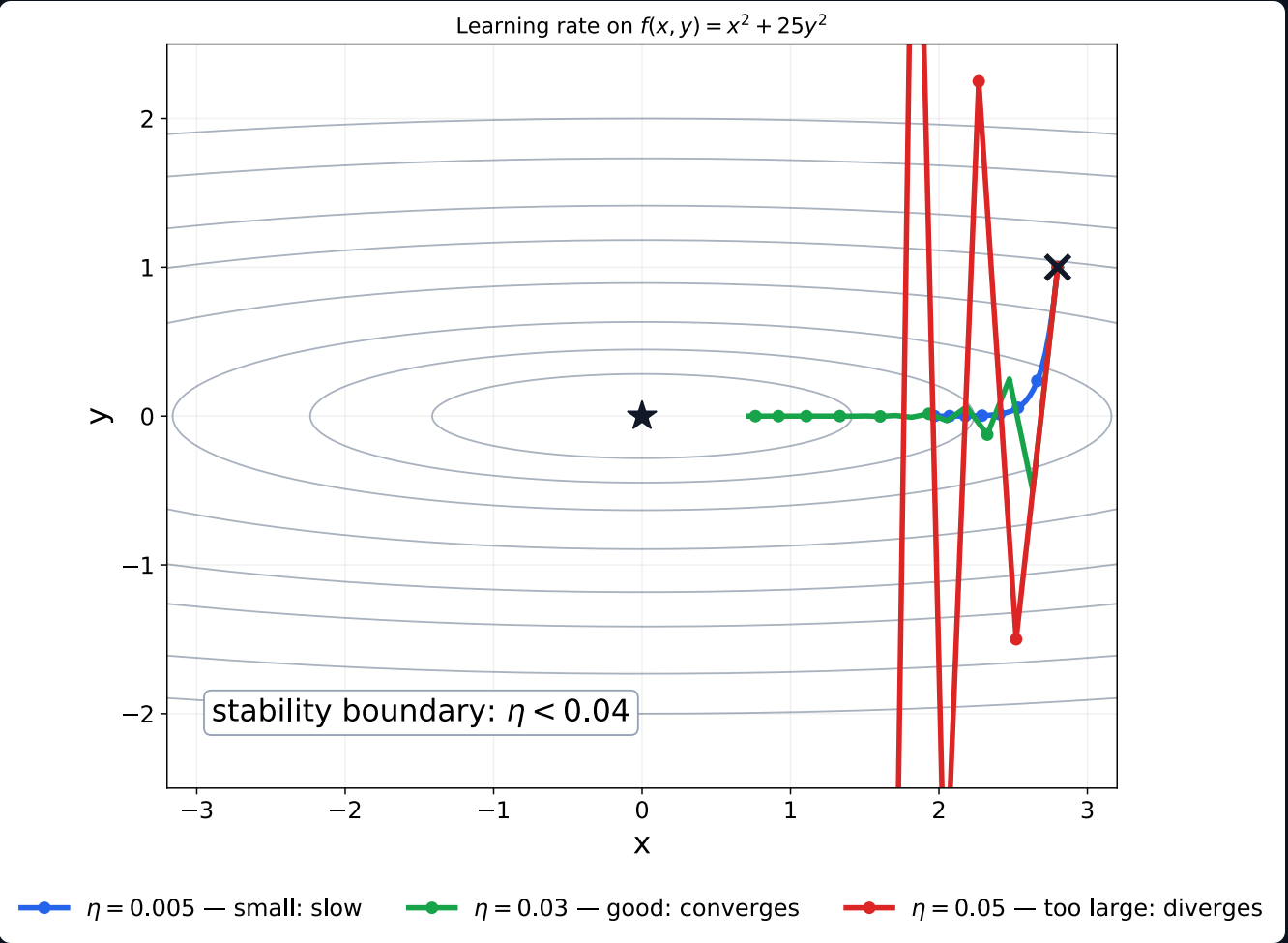


Figure 04: Too small, well-tuned, oscillatory, and diverging LR

Local Minima vs. Saddle Points

1. Local vs Global Minima: $f(x, y) = (x^2 - 1)^2 + 0.2x + y^2$.

- Different basins of attraction trap gradient descent depending on initialization θ_0 .
- Negative- x well is global minimum; positive- x well is a higher local minimum.

2. Saddle Points: $f(x, y) = x^2 - y^2 \implies \nabla f(0, 0) = \mathbf{0}$.

- Hessian has mixed eigenvalues $+2$ and -2 .
- Gradient is zero at $(0, 0)$, but it is *neither* a minimum nor a maximum!

 **Rule:** Zero gradient $\nabla f = 0$ is only a stationary condition, not proof of reaching a minimum.

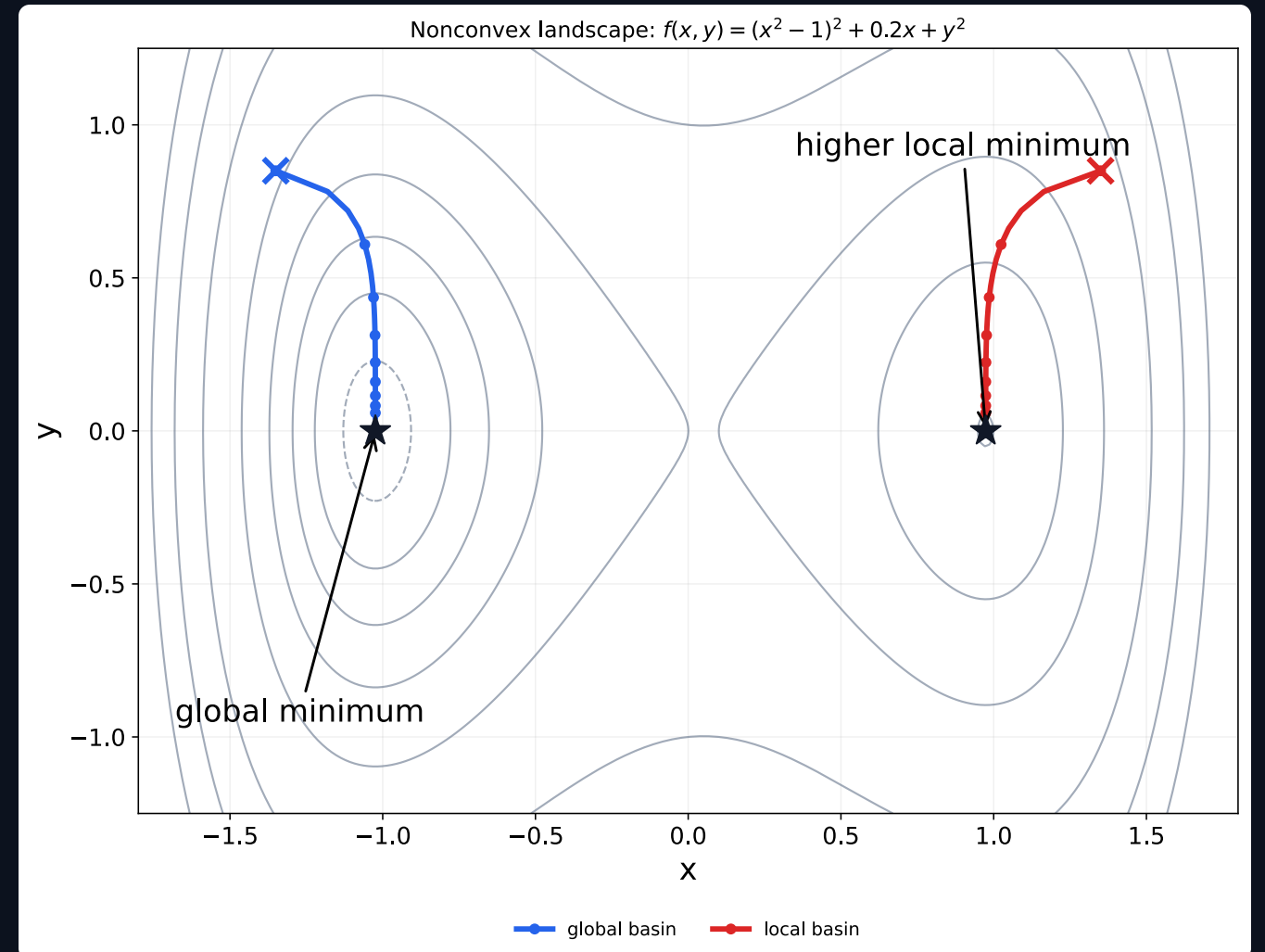


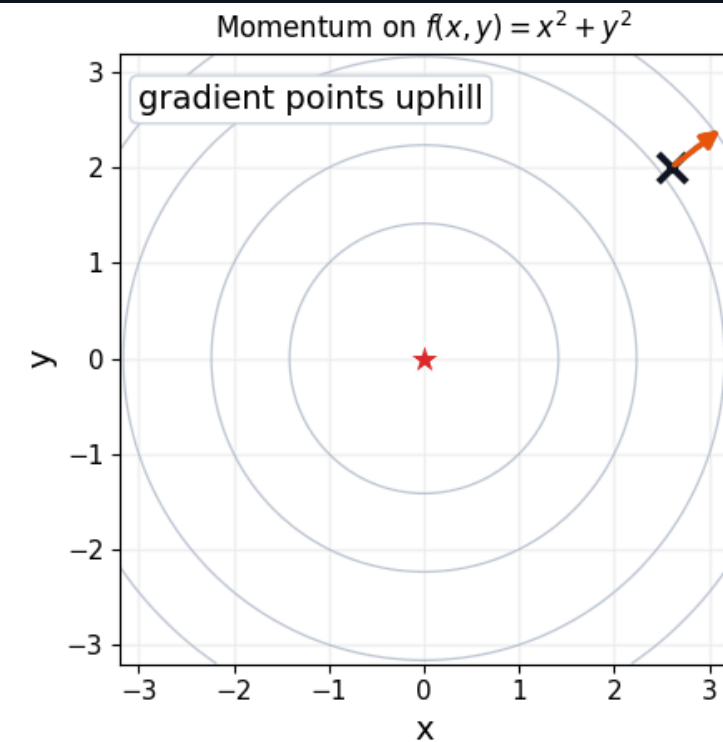
Figure 06: Basin of attraction trapping on nonconvex double-well

Momentum: Accelerating Across Ravines

Motivation: Vanilla GD oscillates wildly across narrow ravines while making agonizingly slow progress along the flat valley.

$$\begin{aligned} v_t &= \beta v_{t-1} + \nabla f(\theta_t) \\ \theta_{t+1} &= \theta_t - \eta v_t \end{aligned}$$

- v_t acts like physical velocity ($\beta \approx 0.9$).
- Oscillating gradient directions cancel out.
- Persistent directions accumulate velocity: effective step size scales up to $\frac{\eta}{1 - \beta}$.



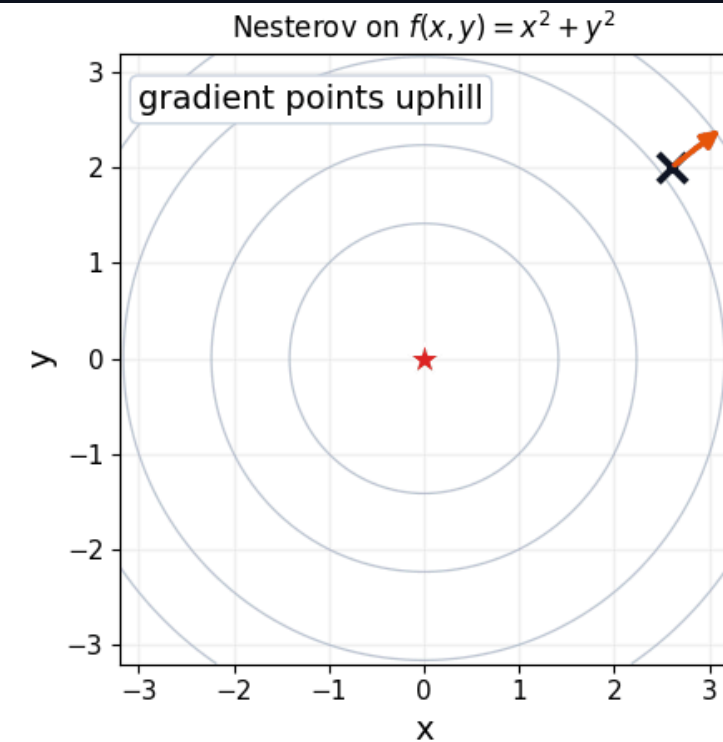
Animated: Momentum dampening oscillations and accelerating downhill

Nesterov: Look-Ahead Before Leaping

Core Idea: Instead of evaluating gradient at the current position, evaluate where momentum is about to carry the parameter:

$$\begin{aligned} v_t &= \beta v_{t-1} + \nabla f(\theta_t - \eta \beta v_{t-1}) \\ \theta_{t+1} &= \theta_t - \eta v_t \end{aligned}$$

- Anticipates future curvature to apply an early braking force.
- Reduces overshooting at the bottom of steep ravines.
- Theoretical convergence on convex quadratics improves from $\mathcal{O}(1/T)$ to $\mathcal{O}(1/T^2)$.



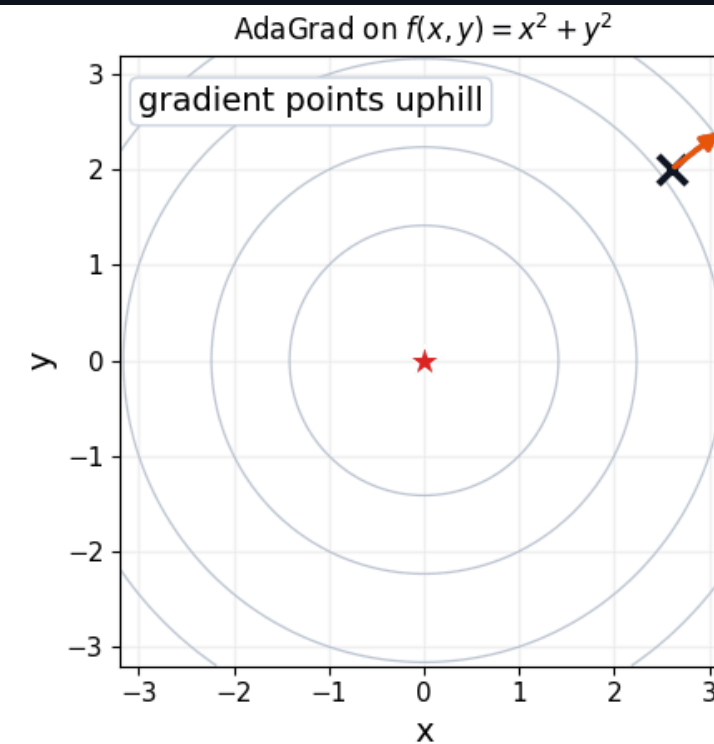
Animated: Nesterov trajectory with predictive look-ahead braking

AdaGrad: Coordinate-Wise Adaptive Scaling

Motivation: Different features have wildly different gradient magnitudes (e.g. rare words vs common words in NLP).

$$G_t = G_{t-1} + g_t \odot g_t \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} \odot g_t$$

- Frequently updated parameters receive smaller effective learning rates.
- Infrequently updated parameters take larger exploratory steps.
- **Limitation:** G_t monotonically grows, causing effective learning rate to decay toward zero prematurely.



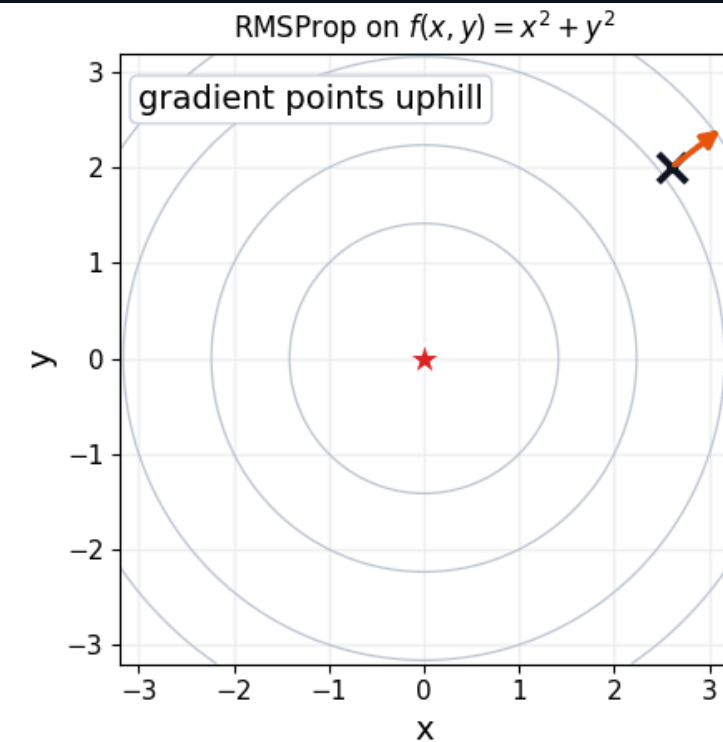
Animated: AdaGrad dampening steps as denominator accumulates

RMSProp: Exponential Moving Average

Fixing AdaGrad: Replace the infinite sum of squares with an exponential moving average (EMA) to forget ancient history:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2 \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \odot g_t$$

- $\rho \approx 0.9$ controls memory window.
- Scale adapts to the *recent* magnitude of gradients.
- Allows continual learning on non-stationary mini-batch objectives.



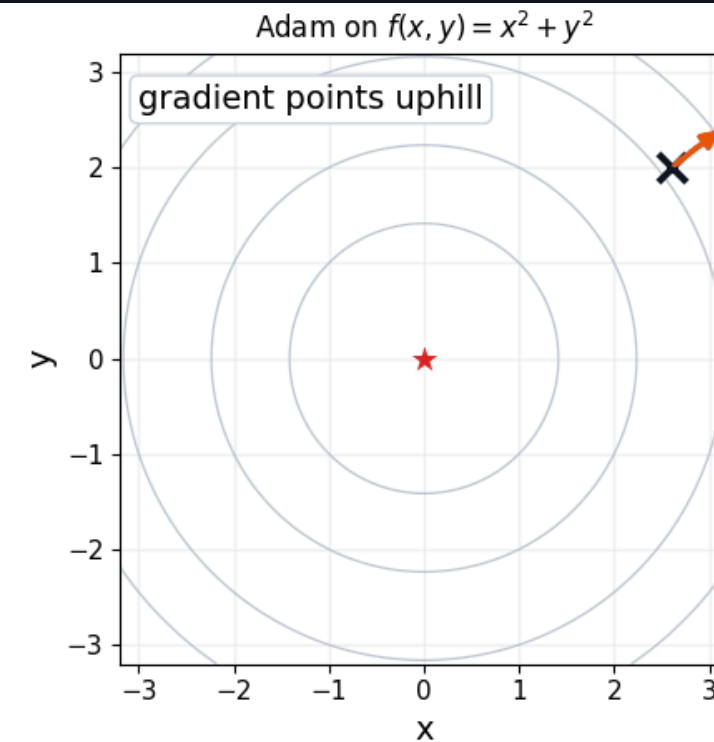
Animated: RMSProp equalizing coordinates on ill-conditioned ravine

Adam: Combining Momentum + RMSProp

Adam tracks both 1st moment (mean) and 2nd moment (uncentered variance):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{aligned}$$

- **Bias correction** ($\hat{m}_t, \hat{v}_t$): Since $m_0 = v_0 = 0$, early steps are biased toward zero. Dividing by $1 - \beta^t$ corrects this early distortion!
- Standard defaults: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.



Animated: Adam smoothly traversing the ill-conditioned surface

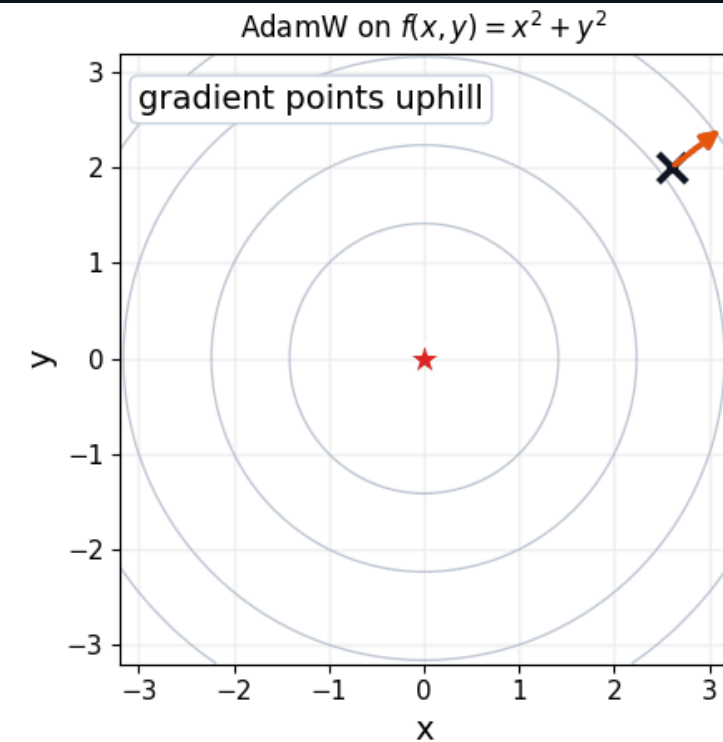
AdamW: Decoupled Weight Decay

L2 Regularization vs Weight Decay:

- In SGD, adding $\frac{1}{2}\lambda \|\theta\|^2$ to loss is mathematically identical to weight decay.
- In Adam, gradients of L2 penalty get scaled inversely by $\sqrt{v_t}$: $\nabla L_{\text{reg}} = g_t + \lambda \theta$ implies $\text{decay} \propto \frac{\lambda \theta}{\sqrt{v_t}}$ Parameters with large historical gradients get *less* regularized!

$$\theta_{t+1} = (1 - \eta \lambda) \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

AdamW restores true weight decay by applying decay *directly to weights* independent of gradient statistics.



Animated: AdamW regularization with decoupled weight shrinkage

Modern Frontiers: Lion & SAM

1. Lion (EvoLved Sign Momentum):

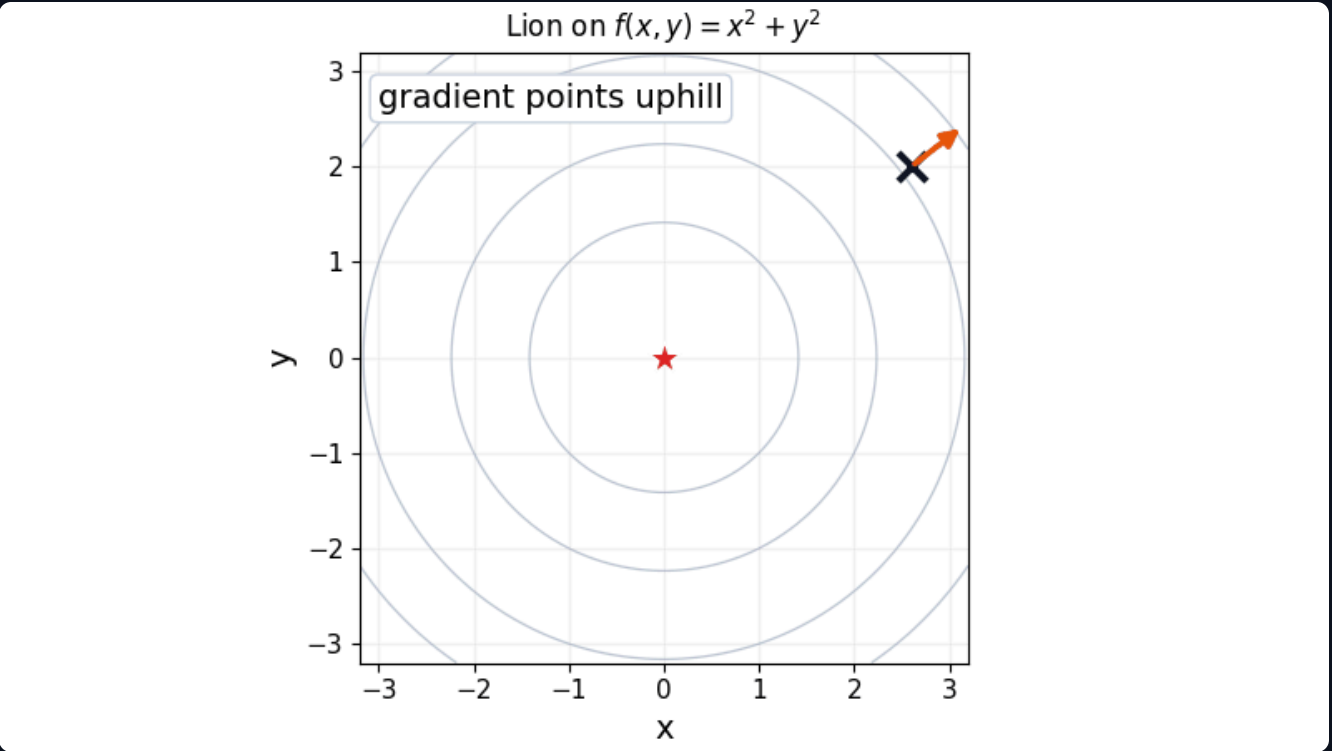
$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_{t+1} = \theta_t - \eta \operatorname{sign}(c_t)$$
$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t$$

Tracks only 1st moment (50% less state memory). Update magnitude is uniform per coordinate via $\operatorname{sign}(\cdot)$.

2. SAM (Sharpness-Aware Minimization):

$$\min_w \max_{\|\epsilon\| \leq \rho} L(w + \epsilon)$$

Searches for flat valleys rather than sharp ravines by calculating gradient at a perturbed worst-case neighbor $w + \rho \frac{\nabla L}{\|\nabla L\|}$.



Animated: Lion making discrete-magnitude sign-based steps

Head-to-Head Trajectory Comparison

Identical Conditions: All 8 optimizers launched from exact same start point $(2.8, 1.0)$ on $f(x, y) = x^2 + 25y^2$:

- **SGD:** Strong oscillations in y before slow progress in x .
- **Momentum & Nesterov:** Momentum carries through oscillations smoothly into the valley.
- **AdaGrad & RMSProp:** Rapidly scale down the steep y direction to equalize motion.
- **Adam & AdamW:** Direct, efficient descent straight to the global minimum $(0, 0)$.
- **Lion:** Distinct staircase trajectory due to $\text{sign}(\cdot)$ operations.

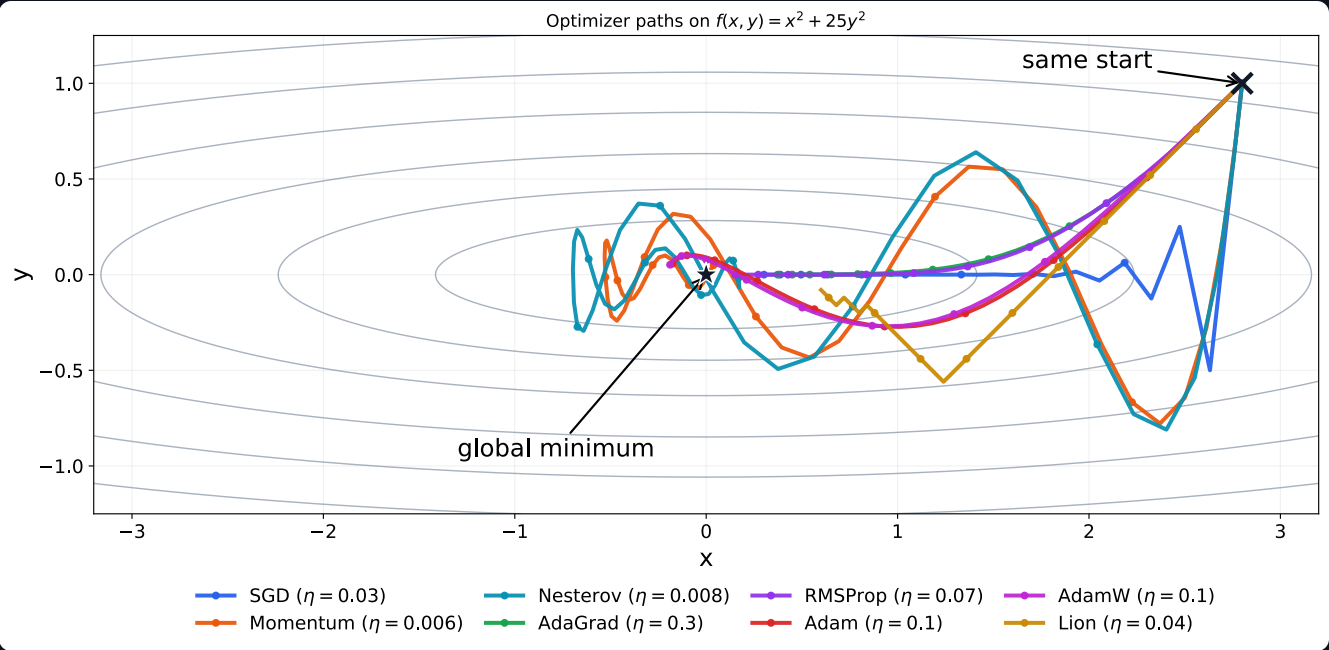


Figure 05: Synchronized trajectories on identical ill-conditioned axes

From 2D Toys to Neural Networks: NumPy CNN

Core Concept: Clean Separation of Concerns

```
$$\text{Forward Pass} \rightarrow \text{Backpropagation} (\nabla_W L) \rightarrow \text{Optimizer.step()}$$
```

- **Network Architecture:**
 - Input: $1 \times 28 \times 28$ MNIST digits.
 - Conv2D + ReLU + MaxPool (16 filters, 3×3).
 - Conv2D + ReLU + MaxPool (32 filters, 3×3).
 - Dense 128 + ReLU + Dropout $\rightarrow$ Dense 10.
- Total trainable parameters: **206,922**.
- Backpropagation provides gradients; optimizers decide the step!

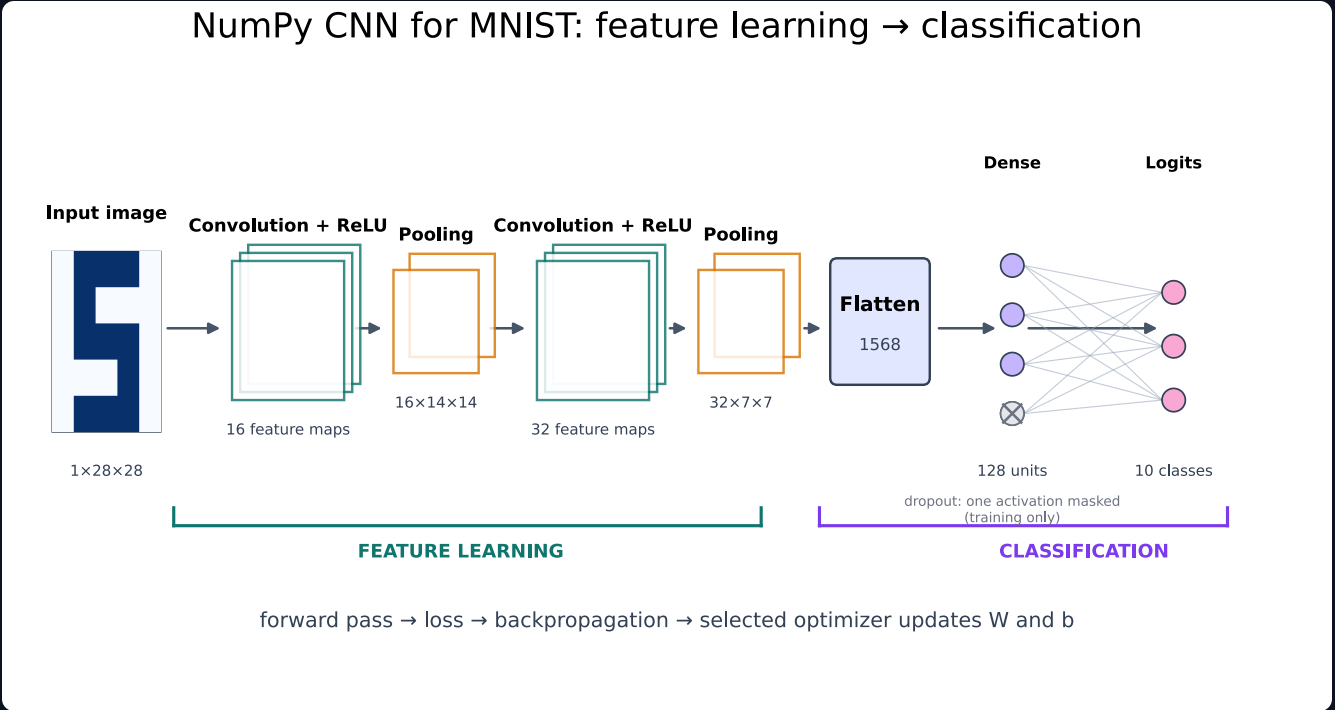
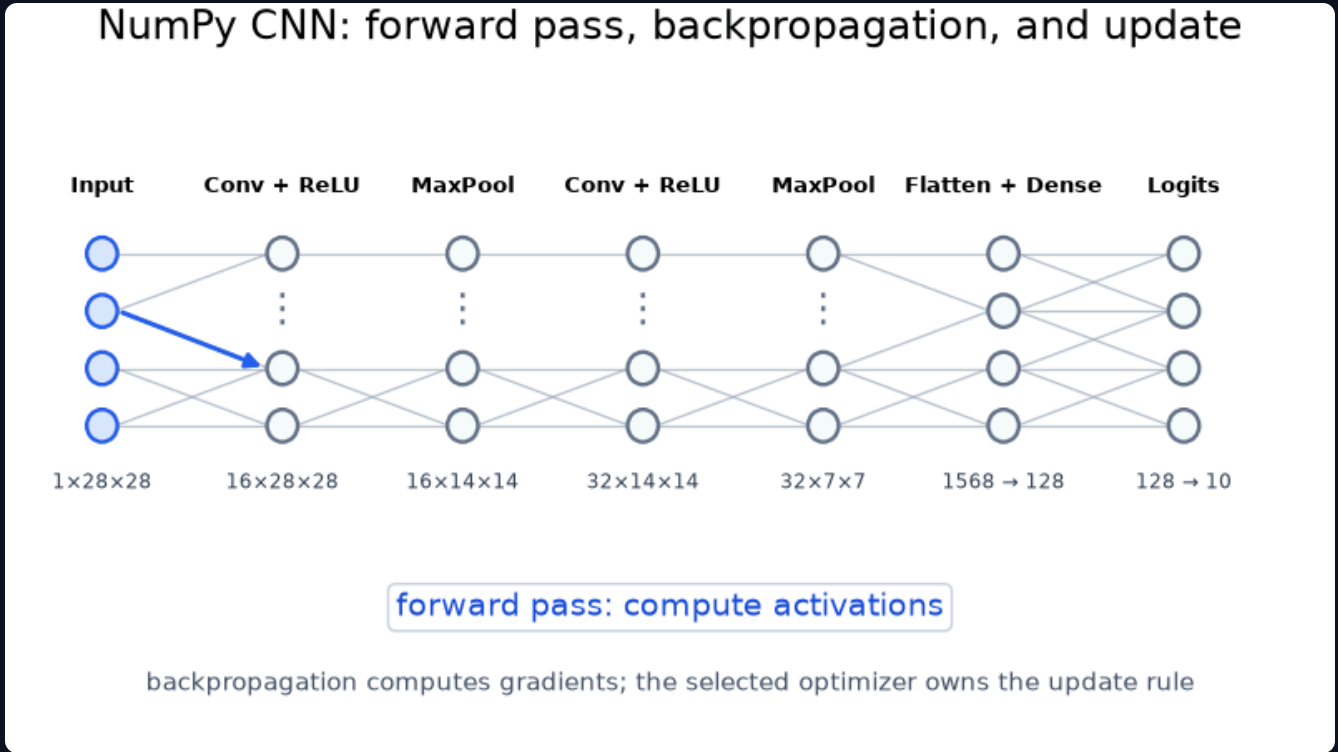


Figure 09: Architectural pipeline of the NumPy CNN

Forward Evaluation vs. Backward Differentiation

The Engine Behind Deep Learning:

- **Forward propagation:** Features flow from left to right through convolutional kernels, spatial pooling, and dense projections to compute cross-entropy loss.
- **Backward propagation:** The scalar loss error signal flows right to left using the multivariable chain rule: $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial W}$
- Optimizers update all 206,922 parameters in-place using their designated algorithm.



Animated: Signal propagation through convolutional & dense layers

Scientific Evaluation Methodology

Fair Comparison Principles:

- **No Test Leakage:** Official test set is evaluated *strictly once* after training finishes; validation split is used for per-epoch monitoring.
- **Controlled Variables:** Same weight initialization seed, same batch sequence, identical parameter budgets.
- **Multiple Seeds:** Report mean and variance across seeds (e.g. seeds 0, 1, 2) rather than cherry-picking one lucky run.
- **Compute Cost Awareness:** Second-order or two-pass methods (like SAM) take roughly $2\times$ the wall-clock time per step.

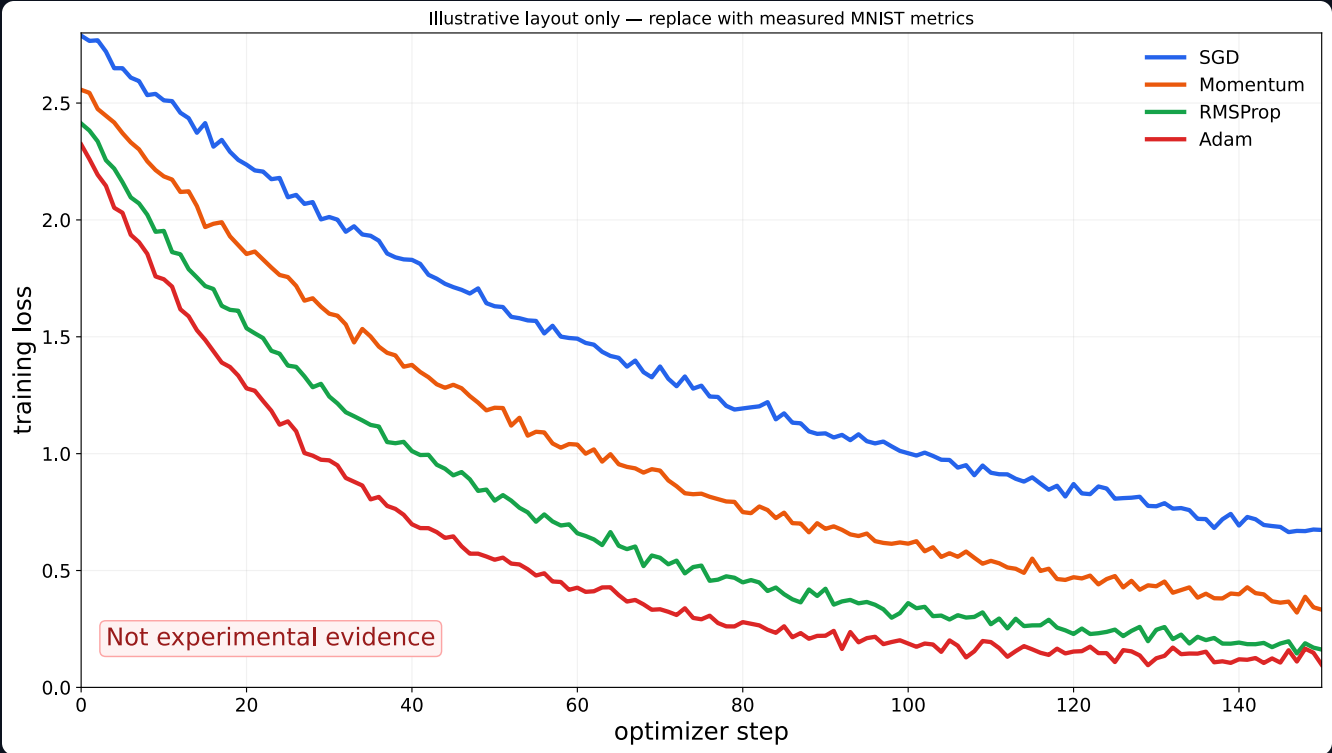


Figure 10: Loss & accuracy tracking curves template

What Every Student Should Remember

1. Calculus Reality: The gradient is a vector of partial derivatives pointing in the direction of steepest local increase.

2. Negative Gradient: Proved downhill via Cauchy-Schwarz and 1st-order Taylor approximation.

3. Curvature Determines Stability: Condition number limits the maximum stable learning rate ($\eta < 2 / \lambda_{\max}$).

4. Local Minima vs Curvature: Ill-conditioned quadratics cause oscillations; nonconvex functions cause basin trapping; saddle points fool the zero-gradient test.

5. Momentum: Smooths oscillations and builds speed across valleys.

6. Adaptive Scaling: Normalizes coordinate disparities; RMSProp bounds history; Adam combines momentum with adaptive variance.

7. No Free Lunch: No optimizer is universally best across all architectures, datasets, and hyperparameters!